

Python을 이용한 파일 처리

03 여러 파일 다루기

이터레이터와 제너레이터



```
list_ = list(range(1, 6))
it = iter(list_)

print(next(it))
# 1

list_.remove(3)

for i in it:
    print(i)
# 2
# 4
# 5
```

- 실제 자료를 저장하지 않고 다음 값을 어떻게 구하는지에 대한 정보만 가지고 있는 자료형
- 실제 자료를 가지고 있지 않으므로 **print** 함수로 이터레이터를 출력하여 값을 확인할 수 없으며 인덱싱 등의 연산도 불가능



```
list_ = list(range(1, 6))
it = iter(list_)

print(next(it))
# 1

list_.remove(3)

for i in it:
    print(i)
# 2
# 4
# 5
```

- **iter** 함수를 이용하여 반복 가능한 자료형을 이터레이터로 변환할 수 있음
- **next** 함수를 이용하여 다음 값을 얻을 수 있음
- **list** 함수를 이용하여 리스트로 변환하거나 **for** 문을 사용하여 반복 가능
- 왼쪽 예와 같이 존재하는 가변 객체를 바탕으로 이터레이터를 생성하면 원본 객체의 변화에 영향을 받음



```
def oscillate_damped(n):
    n_neg = -n
    flag = True

    while n_neg < 0 < n:
        if flag:
            n -= 1
            yield n
        else:
            n_neg += 1
            yield n_neg

        flag ^= True

od = oscillate_damped(2)

for i in od:
    print(i)
    # 1
    # -1
    # 0

next(od)
# StopIteration
```

- 이터레이터를 구현하는 함수
- **return** 대신 **yield**를 사용하여 값을 반환
 - **yield**를 실행하면 **next** 등 다음 값을 요구할 때까지 동작을 멈춤
- 이터레이터에 의해 생성된 항목이 더 없는 경우 **StopIteration** 발생



```
from random import random

# 임의의 값을 무한히 반복하는 이터레이터
random_iterator = iter(random, None)
list(random_iterator) # 오류는 없으나 수행 불가능

---

import sys

def my_generator():
    """1, 2, 1, 2, 3, 2, 3, 4, ..., 2**63 - 2, 2**63 - 1"""
    # 2**63 - 1 == 9223372036854775807
    x = 0
    while x <= sys.maxsize:
        for _ in range(2):
            x += 1
            yield x
        x -= 1
        yield x

it = my_generator()
list(it) # 오류는 없으나 수행 불가능
```

- 이터레이터 및 제너레이터는 단순히 다음 값을 어떻게 구하는지에 대해서만 정의
 - 단순히 정의만 할 때는 list와 같이 실제 자료가 필요한 자료형에 비해 시간과 메모리가 절약됨
 - 매우 크거나 끝나지 않는 반복의 경우 list 등의 자료형으로 정의 불가능
- range와 비슷하지만 range보다 복잡한 설계 가능
- 하위 폴더가 많은 위치("C:\" 및 '/' 등)에서 walk 메서드를 호출하면 매우 큰 목록이 만들어질 수 있음
 - 이터레이터를 반환하면 메서드 호출에 대한 오버헤드를 줄일 수 있음